

AI Usage and Evidence Report

C Shell Mini-Project (CS3301 / OSN)

Purpose: documentary evidence of how AI assistance was used during development, based only on the conversation, files, generated artifacts, and test outputs available in this ChatGPT project context.

Evidence standard used in this report

This report deliberately separates documented facts from interpretation. Where the conversation or file record clearly establishes that an artifact was generated by AI, that is stated directly. Where only the assignment specification or a user-uploaded source file is available, it is not relabeled as AI-generated. Where an exact earlier prompt is not preserved in the accessible evidence, the report says so instead of reconstructing a fictional prompt.

Scope: Sections A–F, followed by G (overall summary), H (evidence index), and I (AI usage declaration).

Prepared from the project conversation and associated conversation files available in ChatGPT.

Evidence Methodology and Source Boundaries

The project context contains the assignment specification, uploaded project files, AI-generated file artifacts, user-provided terminal outputs, and an iterative record of implementation/debugging. The assignment specification itself identifies Parts A–F and their requirements. The conversation file inventory also records whether individual artifacts were uploaded by the user or generated in the conversation. This distinction is important for an AI-use report: a file being present in the conversation is not, by itself, evidence that AI wrote it.

The strongest direct evidence of AI generation is the file metadata marking artifacts as `<b>source_kind: generated</b>`. Examples include `executor_modified.c`, `executor_compact.c`, `executor_d1.c`, `executor_d2.c`, `executor_e1.c`, `executor_e1_modular.zip`, and the generated `executor/executor_utils/redirection/process/jobcontrol` set. The initial `lexer`, `parser`, `prompt`, `built-in`, `main`, and `executor` files were recorded as user uploads. Therefore this report describes those initial files as the project baseline supplied to the conversation, not as AI-generated artifacts.

The later F1/F2 work is supported especially strongly by the actual interactive debugging transcript. The conversation records the user installing a downloaded F1 patch, reporting a segmentation fault, applying an AI-suggested safer implementation, testing `spy`, then receiving a generated F2 patch containing `snoop.c`, `snoop.h`, and `executor_f2.c`. The user then compiled, ran, reported failures, and applied targeted corrections suggested by AI. This provides unusually concrete evidence of an AI-assisted development loop.

For A–D, the accessible evidence is less granular about the exact wording of the original user prompts. Consequently, the report discusses the code artifacts and the generated executor variants that are actually recorded, but it does not invent quotations such as “write my entire Part A” when no such exact message is available. The same rule is applied to E where an exact E4 generated-file artifact is not present in the current file inventory, even though the conversation records the implementation and tests.

Evidence references in the report use the conversation/file identifiers available in the ChatGPT file context, such as the assignment file `Pasted markdown(4).md`, the uploaded project baseline `mini-project1.7z` and source files, and generated artifacts with their recorded user-turn indices. Terminal outputs are identified by the actual commands and outputs appearing in the conversation.

Part A — Shell Input

Evidence status: Implementation/file evidence is available; exact early prompts and/or complete runtime transcripts are not always preserved.

The assignment defines Part A as the base layer of the shell. It requires a prompt showing username, hostname, and current path; interactive input; and lexical plus grammar validation before execution. The specification explicitly defines token classes for pipes, ampersands, semicolons, redirections, words, quoting, and escapes, and requires invalid syntax to be rejected before execution. The assignment also states that the lexer/parser follow a regular grammar and encourages the use of automata-theory ideas. These requirements make Part A foundational because later parts depend on correctly turning raw input into structured commands.

The available project files show a modular implementation rather than a monolithic shell. The baseline contains `main.c`, `prompt.c`, `lexer(1).c`, `parser.c` and corresponding headers. The main loop obtains a line with `getline()`, removes the newline, tokenizes it, parses the token stream, executes a parsed command group, and frees the associated structures. `prompt.c` obtains the hostname, user name, current directory and startup home directory, and formats the home prefix as `~`. `lexer(1).c` recognizes shell operators and quoted/escaped words. `parser.c` constructs `ParsedCommandGroup` and `CommandPipeline` structures and records redirections.

AI involvement in this section must be described carefully. The initial A-related source files in the conversation were uploaded by the user at the beginning of the project context, so their presence is not evidence that AI generated those initial files. However, the conversation later shows AI being used to modify and regenerate executor-related files that consume the parser's output. The file inventory records generated executor variants at later user turns. Thus, the defensible statement is that AI participated in the broader shell architecture and integration around Part A, while the exact original A-specific prompt is not preserved in the evidence available here.

1. Section overview and project role

The assignment specification assigns this part a defined set of behaviors and constraints. In the overall shell, the part is not isolated: it consumes parser structures, depends on executor behavior, and can affect later process-management features. The implementation evidence therefore has to be read as part of the evolving shell architecture rather than as a collection of independent snippets.

The AI-use question is separate from the implementation question. A source file may be correct without being AI-generated, and an AI-generated file may require substantial human correction before it becomes usable. This report keeps those questions separate throughout.

2. AI involvement and available prompt evidence

The user stated that the project was developed with generous AI use and that the dominant interaction style was to provide requirements/specification and ask ChatGPT to generate required files directly. The accessible project evidence supports that general description through multiple generated artifacts. However, for several early sections the exact original prompt text is not present in the retrievable conversation record. For those cases, this report does not fabricate a quotation. The evidence is instead described using the actual generated-file metadata and the source code that resulted.

Where a prompt is directly visible in the current conversation, it is quoted only when it actually appears. For F2, for example, the conversation contains explicit requests such as “edit this” followed by the full `snoop.c` source, and later explicit testing instructions and results. For earlier sections, the evidence is weaker at the prompt-text level and stronger at the artifact level.

Part A — Shell Input — Implementation and AI-Generated Work

3. AI-generated implementation

The prompt implementation is a direct realization of the specification. It calls `gethostname()` and `getpwuid(getuid())` to obtain identity information, then `getcwd()` to obtain the current directory. It compares the current directory with `shell_ctx.startup_home_directory`. If they are equal, it prints `~`; if the current directory is below the startup home, it replaces the home prefix with `~`; otherwise it prints the absolute path. This directly corresponds to the A1 requirements.

The main input loop is visible in `main.c`. `init_shell()` records the startup directory as the shell home. The loop calls `reap_background_processes()`, displays the prompt, calls `getline()`, handles EOF, strips the newline, ignores an empty line, tokenizes the input, parses it, logs it when parsing succeeds, executes the group, and frees the structures. This sequence is important evidence that input is consumed before execution and that the parser is placed between input acquisition and execution.

The lexer recognizes whitespace as separators, treats `;`, `&`, `|`, `<`, `>` and `>>` as operators, and constructs `WORD` tokens from ordinary characters, escapes and quoted bodies. The visible implementation handles both single and double quotes and contains special escape processing. The parser then consumes tokens into command structures and stores input/output redirections. The generated/available source therefore demonstrates the intended division between lexical analysis and grammar-level command representation.

4. How the implementation maps to the specification

The implementation should be understood as a mapping from explicit requirements to concrete data structures, system calls and control flow. The strongest examples in the available evidence are the parser structures in A, the built-in source files in B, the executor/redirection infrastructure in C, generated D1/D2 executor stages, generated E1 process/job-control infrastructure, and the F1/F2 patches. In each case, the AI contribution is best described as generating or revising concrete implementation artifacts against a supplied specification.

A significant benefit of AI in this setting was the ability to produce complete multi-file changes rather than isolated code fragments. The conversation file inventory records generated artifacts at multiple stages, including executor variants and a modular package. This matters because the assignment explicitly penalizes monolithic code and expects functionality to be separated across `.c/.h` files.

At the same time, the implementation evidence contains imperfections. The parser source visible in the conversation contains an “Invalid syntax!” message even though the assignment requires “cshell: invalid syntax”. The initial snoop implementation failed to compile because `environ` was undeclared and later had runtime `ptrace` issues. These concrete imperfections demonstrate why generated code still requires engineering review.

Part A — Shell Input — Development Workflow and Verification

5. Development workflow

The recorded workflow begins with the assignment specification and a project archive/source files being supplied to ChatGPT. The conversation file inventory shows the baseline project archive and source/header files were uploaded together, including `lexer(1).c`, `parser.c`, `main.c`, `prompt.c`, `executor.c` and the built-in implementations. This establishes that AI was given the project context rather than asked to design a shell in isolation.

Subsequent generated artifacts show that AI was used to work on executor integration in stages. The file inventory records `executor_modified.c`, `executor_compact.c` and later `executor_d1.c` and `executor_d2.c` as generated artifacts. This is concrete evidence that AI was producing implementation files during the project's development, even where the exact user prompt leading to a particular artifact is no longer visible.

The user then compiled the resulting project and supplied compiler/runtime output back to ChatGPT. For later features, this pattern is explicit: the user reports exact terminal diagnostics, and the AI diagnoses them. The same project structure is retained, with changes being integrated into `src/` and `include/`. This shows AI was used as an iterative engineering assistant rather than only as a source of conceptual explanations.

6. Verification and testing

Direct A-specific terminal tests are not fully preserved in the accessible conversation. Therefore this report does not claim that every A1–A3 example was executed. What can be verified from the source artifacts is that `main.c` implements `getline`-based input, `prompt.c` implements the specified prompt transformation, `lexer(1).c` implements operator/word scanning, and `parser.c` creates command groups and pipelines.

The assignment itself requires empty/whitespace input to be accepted and invalid grammar to produce `cshell: invalid syntax`. The source parser visible in the conversation contains a notable message, `fprintf(stderr, "Invalid syntax!")`, rather than the exact required text in the specification. This is precisely the kind of distinction that an evidence report should preserve rather than silently declaring full compliance.

Accordingly, the verification status for Part A in this report is: implementation evidence is available; comprehensive runtime evidence is not available in the conversation. Screenshots of the relevant source files and the initial project upload are stronger evidence than invented test claims.

The central workflow visible in the conversation is therefore iterative rather than linear: requirements/context were supplied; AI produced implementation artifacts; the user integrated them; the project was compiled; runtime behavior was observed; actual errors were reported to AI; AI diagnosed and supplied a correction; and the user retested. F2 provides the clearest complete example of this cycle.

Part A — Shell Input — Contributions, Evidence, Reflection and Screenshots

7. My contribution vs AI contribution

User contribution documented: supplied the assignment/project context and baseline files; integrated files into the project; issued compilation and runtime commands throughout the project; supplied actual compiler and runtime output; and made shell-side edits where the transcript explicitly shows commands such as `sed -i`.

AI contribution documented: generated later executor variants and provided implementation/debugging guidance. The file inventory explicitly labels several executor artifacts as generated. The conversation also shows AI analyzing later compiler and runtime output and proposing code changes.

Not claimed: that AI wrote the original lexer, parser, prompt, or main files merely because those files were discussed. Those were recorded as user-uploaded artifacts. Exact original A-specific prompts are marked as unavailable rather than reconstructed.

8. Evidence available for submission

Assignment: Pasted markdown(4).md, lines 45–225, defines A1–A3, lexer tokens, quoting, grammar, validation and syntax-error requirements. ■filecite references in the chat point to this file.

Baseline project files: conversation file inventory records lexer(1).c, parser.c, main.c and prompt.c as user uploads at user turn 2.

Source evidence: main.c shows `getline()`, `tokenize_input_line()`, `parse_command_grammar()`, `execute_command_group()` and `cleanup`. prompt.c shows `gethostname()`, `getpwuid()`, `getcwd()` and `~` conversion. lexer(1).c shows operator/word tokenization. parser.c shows `ParsedCommandGroup/CommandPipeline` construction.

Generated-artifact evidence: `executor_modified.c` and `executor_compact.c` are marked `source_kind: generated` in the conversation file inventory.

9. Reflection

Part A illustrates both the usefulness and limitation of AI evidence. AI could operate on the supplied project structure and generate downstream executor variants, but the available record does not preserve every early conversational instruction. A truthful report therefore treats source metadata and visible code as evidence, not as proof of an unrecorded prompt. The most important human responsibility remained integrating the generated artifacts and checking whether their behavior and error messages actually matched the specification.

10. Suggested screenshots

- Assignment Part A page showing A1–A3 requirements and grammar.
- Conversation file list showing main.c, prompt.c, lexer(1).c and parser.c as uploaded baseline files.
- main.c source showing the input/tokenize/parse/execute loop.
- prompt.c source showing startup-home handling and prompt formatting.
- lexer(1).c source showing tokenization of `|`, `<`, `>`, `>>`, quotes and escapes.
- parser.c source showing command-group/pipeline construction.

11. ChatGPT conversation link placement

For submission, the ChatGPT conversation link should be placed beside the evidence index entry for this section. The link should point to the actual conversation containing the relevant prompt/response and, where possible, the generated file card or debugging exchange. Do not create a separate link claiming to represent a conversation that does not exist. For early sections where the exact prompt is unavailable in the current evidence, the report should rely on the actual conversation/file artifact that is available.

Part B — Shell Intrinsic

Evidence status: Implementation/file evidence is available; exact early prompts and/or complete runtime transcripts are not always preserved.

Part B contains the shell's built-in commands: hop, reveal, peek and locate. Together they provide directory navigation, directory listing, file-content inspection and executable discovery. The assignment explicitly bans `exec*` syscalls for Part B, so these commands are intended to operate inside the shell rather than being delegated to external processes.

The assignment gives detailed behavioral rules. hop changes the shell's working directory and maintains a persistent deterministic frequency record. reveal lists directory contents, supports `-a` and `-t`, sorts lexicographically and resolves paths similarly to hop without frequency fallback. peek supports `-n` and `-r`, including backwards chunked reading with `lseek()` for regular files. locate searches the current directory and PATH and prints absolute executable paths.

The available baseline source files correspond directly to these four intrinsics: `builtin_hop.c`, `builtin_reveal.c`, `builtin_peek.c` and `builtin_locate.c`. The conversation file inventory records them as user-uploaded baseline files. AI involvement is evidenced more strongly in later executor-generation work that integrates built-ins than by a preserved Part-B prompt. The report therefore avoids inventing a claim that the initial built-in source was generated by AI.

1. Section overview and project role

The assignment specification assigns this part a defined set of behaviors and constraints. In the overall shell, the part is not isolated: it consumes parser structures, depends on executor behavior, and can affect later process-management features. The implementation evidence therefore has to be read as part of the evolving shell architecture rather than as a collection of independent snippets.

The AI-use question is separate from the implementation question. A source file may be correct without being AI-generated, and an AI-generated file may require substantial human correction before it becomes usable. This report keeps those questions separate throughout.

2. AI involvement and available prompt evidence

The user stated that the project was developed with generous AI use and that the dominant interaction style was to provide requirements/specification and ask ChatGPT to generate required files directly. The accessible project evidence supports that general description through multiple generated artifacts. However, for several early sections the exact original prompt text is not present in the retrievable conversation record. For those cases, this report does not fabricate a quotation. The evidence is instead described using the actual generated-file metadata and the source code that resulted.

Where a prompt is directly visible in the current conversation, it is quoted only when it actually appears. For F2, for example, the conversation contains explicit requests such as “edit this” followed by the full `snoop.c` source, and later explicit testing instructions and results. For earlier sections, the evidence is weaker at the prompt-text level and stronger at the artifact level.

Part B — Shell Intrinsics — Implementation and AI-Generated Work

3. AI-generated implementation

`builtin_hop.c` contains a persistent frequency database named `.cshell_freqency`, with entries containing a path, score and `last_accessed` timestamp. Successful hops update the score and timestamp. Effective score calculation incorporates age, and exact basename matches are weighted much more strongly than substring matches. The source also supports special targets such as `~`, `.`, `..` and `-`, and stores the startup home directory through `shell_ctx`.

`builtin_peek.c` demonstrates the assignment's two input strategies. For ordinary streams it uses `getline()` to collect lines. For reverse reading of regular files it opens the file, uses `lseek()` to obtain the file size, moves backwards through fixed-size chunks, reconstructs lines and then prints them in reverse order. The visible source explicitly defines `CHUNK_SIZE` as 4096 and contains the chunk-processing logic. This is concrete implementation evidence for the `lseek`-based reverse-reading requirement.

The executor baseline includes built-in dispatch through comparisons for `hop`, `reveal`, `peek`, `locate` and `log`. It also contains a command-resolution function that handles literal paths, current-directory lookup and `PATH` lookup, which is relevant to `locate/command` execution integration. Generated executor variants later modified this infrastructure as new sections were added.

4. How the implementation maps to the specification

The implementation should be understood as a mapping from explicit requirements to concrete data structures, system calls and control flow. The strongest examples in the available evidence are the parser structures in A, the built-in source files in B, the executor/redirection infrastructure in C, generated D1/D2 executor stages, generated E1 process/job-control infrastructure, and the F1/F2 patches. In each case, the AI contribution is best described as generating or revising concrete implementation artifacts against a supplied specification.

A significant benefit of AI in this setting was the ability to produce complete multi-file changes rather than isolated code fragments. The conversation file inventory records generated artifacts at multiple stages, including executor variants and a modular package. This matters because the assignment explicitly penalizes monolithic code and expects functionality to be separated across `.c/.h` files.

At the same time, the implementation evidence contains imperfections. The parser source visible in the conversation contains an “Invalid syntax!” message even though the assignment requires “cshell: invalid syntax”. The initial snoop implementation failed to compile because `environ` was undeclared and later had runtime `ptrace` issues. These concrete imperfections demonstrate why generated code still requires engineering review.

Part B — Shell Intrinsics — Development Workflow and Verification

5. Development workflow

The available conversation file inventory shows that the user first uploaded the baseline built-in source/header files and project archive. The later generated executor artifacts indicate AI-assisted modification of the execution layer over several stages: `executor_modified.c`, `executor_compact.c`, `executor_d1.c`, `executor_d2.c` and `executor_e1.c` were generated in the conversation.

Because the exact original Part-B prompts are not retained in the evidence available to this report, it would be inaccurate to write invented quotations such as 'generate builtin_hop.c'. Instead, the defensible description is that the user supplied the project and requirements, and AI was used extensively in subsequent implementation/integration work while the original Part-B source artifacts remained identified as uploads.

The user's role in this stage included integrating generated files, compiling the shell, and later carrying out real tests for process-management features that depended on the built-in/executor infrastructure. This demonstrates that the generated code was not treated as self-validating.

6. Verification and testing

Source-level verification is strong for B. `builtin_hop.c` visibly contains persistent storage, score updates and decay-style effective scoring. `builtin_peek.c` visibly uses `lseek()` for reverse regular-file processing and handles missing files/directories with the specified message family. The assignment source gives the expected semantics and examples.

Comprehensive command-by-command runtime output for `hop`, `reveal`, `peek` and `locate` is not preserved in the accessible conversation record used for this report. Therefore the report does not claim that every B1–B4 example was tested during the recorded conversation.

The evidence should be presented as implementation evidence plus any local screenshots the student captured while developing. If a screenshot of a successful Part-B command test exists outside the current conversation, it can be added to the submission, but it should not be described here as a conversation-derived result.

The central workflow visible in the conversation is therefore iterative rather than linear: requirements/context were supplied; AI produced implementation artifacts; the user integrated them; the project was compiled; runtime behavior was observed; actual errors were reported to AI; AI diagnosed and supplied a correction; and the user retested. F2 provides the clearest complete example of this cycle.

Part B — Shell Intrinsics — Contributions, Evidence, Reflection and Screenshots

7. My contribution vs AI contribution

User: supplied the project files/specification and integrated the resulting artifacts into the shell. The initial built-in files are explicitly recorded as user uploads.

AI: generated multiple executor variants and assisted with the evolving integration architecture. The generated executor artifacts are concrete evidence of AI code-generation activity around the built-ins.

Verification: the user was responsible for running the shell/build process and observing outputs. The record does not support attributing all original built-in implementation to AI.

8. Evidence available for submission

Assignment Part B: Pasted markdown(4).md, lines 227–438, contains B1–B4 requirements.

builtin_hop.c is recorded as a user-uploaded conversation file; source contains .cshell_freecy, score and last_accessed.

builtin_peek.c is recorded as a user-uploaded conversation file; source contains lseek(), fixed-size chunk reading and -n/-r handling.

executor_modified.c, executor_compact.c and later executor_d1/d2/e1 are recorded as generated artifacts, evidencing AI-generated integration work.

9. Reflection

Part B is a useful example of why an AI-usage report should distinguish 'AI was used on the project' from 'AI wrote every line'. The file metadata supports the former strongly and the latter only for specific generated artifacts. The project demonstrates a mixed workflow: an existing baseline was supplied, AI generated successive executor/integration artifacts, and the user incorporated and validated them.

10. Suggested screenshots

- Assignment Part B requirements.
- Conversation file inventory showing built-in source files as uploaded.
- builtin_hop.c freecy structures and update logic.
- builtin_peek.c lseek/chunk reverse-reading logic.
- Generated executor_modified.c or executor_compact.c file card showing source_kind: generated.
- Any actual terminal test screenshot for a B command, if available in the chat/project.

11. ChatGPT conversation link placement

For submission, the ChatGPT conversation link should be placed beside the evidence index entry for this section. The link should point to the actual conversation containing the relevant prompt/response and, where possible, the generated file card or debugging exchange. Do not create a separate link claiming to represent a conversation that does not exist. For early sections where the exact prompt is unavailable in the current evidence, the report should rely on the actual conversation/file artifact that is available.

Part C — File Redirection and Pipes

Evidence status: Implementation/file evidence is available; exact early prompts and/or complete runtime transcripts are not always preserved.

Part C covers command execution, input redirection, output redirection and pipelines. It is a core operating-systems portion of the shell because it requires direct use of `fork()`, `pipe()`, `open()`, `read()`, `write()`, `dup2()`, `close()` and exec-related process execution rather than `system()` or `popen()`. The specification also requires special command resolution rules, multiple redirections, and continued execution of other pipeline stages when one stage cannot start.

The baseline executor source shows substantial implementation infrastructure for these requirements. It contains command resolution, input-redirection handling, output-redirection handling, pipeline creation, child creation and built-in execution. Later generated executor artifacts show that AI was used to evolve this execution layer as the project progressed.

The exact initial Part-C AI prompt is not preserved in the evidence available here. Therefore this section documents the actual code artifacts and the later generated executor work rather than manufacturing a verbatim prompt.

1. Section overview and project role

The assignment specification assigns this part a defined set of behaviors and constraints. In the overall shell, the part is not isolated: it consumes parser structures, depends on executor behavior, and can affect later process-management features. The implementation evidence therefore has to be read as part of the evolving shell architecture rather than as a collection of independent snippets.

The AI-use question is separate from the implementation question. A source file may be correct without being AI-generated, and an AI-generated file may require substantial human correction before it becomes usable. This report keeps those questions separate throughout.

2. AI involvement and available prompt evidence

The user stated that the project was developed with generous AI use and that the dominant interaction style was to provide requirements/specification and ask ChatGPT to generate required files directly. The accessible project evidence supports that general description through multiple generated artifacts. However, for several early sections the exact original prompt text is not present in the retrievable conversation record. For those cases, this report does not fabricate a quotation. The evidence is instead described using the actual generated-file metadata and the source code that resulted.

Where a prompt is directly visible in the current conversation, it is quoted only when it actually appears. For F2, for example, the conversation contains explicit requests such as “edit this” followed by the full `snoop.c` source, and later explicit testing instructions and results. For earlier sections, the evidence is weaker at the prompt-text level and stronger at the artifact level.

Part C — File Redirection and Pipes — Implementation and AI-Generated Work

3. AI-generated implementation

The executor source contains `resolve_binary_path()`, which distinguishes a command containing `/`, a `%` prefix, a current-directory executable and `PATH` lookup. This maps directly to C1's unusual command-resolution rules. The function uses `stat()` and `access(X_OK)` to verify executable files.

The baseline executor's input redirection uses a pipe as an internal stream: it opens each requested file with `O_RDONLY`, reads data into a buffer and writes the bytes into the pipe, then `dup2()`s the read end onto `STDIN_FILENO`. This supports multiple input files in sequence and closes descriptors after use.

Output redirection is implemented with a relay process. The command output is connected to a pipe; the relay opens each output file using `O_WRONLY|O_CREAT` and either `O_APPEND` or `O_TRUNC`, then copies data from the pipe to every destination. The visible code uses 0644 creation permissions and closes the descriptors. This directly reflects the requirement that multiple output targets each receive the complete command output.

4. How the implementation maps to the specification

The implementation should be understood as a mapping from explicit requirements to concrete data structures, system calls and control flow. The strongest examples in the available evidence are the parser structures in A, the built-in source files in B, the executor/redirection infrastructure in C, generated D1/D2 executor stages, generated E1 process/job-control infrastructure, and the F1/F2 patches. In each case, the AI contribution is best described as generating or revising concrete implementation artifacts against a supplied specification.

A significant benefit of AI in this setting was the ability to produce complete multi-file changes rather than isolated code fragments. The conversation file inventory records generated artifacts at multiple stages, including executor variants and a modular package. This matters because the assignment explicitly penalizes monolithic code and expects functionality to be separated across `.c/.h` files.

At the same time, the implementation evidence contains imperfections. The parser source visible in the conversation contains an "Invalid syntax!" message even though the assignment requires "cshell: invalid syntax". The initial snoop implementation failed to compile because `environ` was undeclared and later had runtime `ptrace` issues. These concrete imperfections demonstrate why generated code still requires engineering review.

Part C — File Redirection and Pipes — Development Workflow and Verification

5. Development workflow

The conversation file inventory shows a sequence of generated executor artifacts over the project: `executor_modified.c`, `executor_compact.c`, `executor_d1.c` and `executor_d2.c`. Later, a generated modular package contains `executor.c`, `executor.h`, `executor_utils.c/h`, `redirection.c/h`, `process.c/h` and `jobcontrol.c/h`. This is strong evidence that AI was used to refactor and extend the execution layer rather than merely answer conceptual questions.

The user integrated these generated artifacts into the actual shell project and compiled them with the project's strict flags. The later F2 work provides explicit examples of the same pattern: AI-generated executor code was copied into `src/executor.c`, then compiler errors were reported back and corrected.

Because the original Part-C conversation prompt is not visible in the currently retrievable transcript, this report intentionally describes it as 'requirements supplied to AI and generated executor work' rather than quoting a prompt that cannot be verified.

6. Verification and testing

Source evidence verifies use of `pipe()`, `fork()`, `dup2()`, `open()`, `read()`, `write()` and `close()` in the relevant executor/redirection logic. The pipeline code creates one pipe per separator and forks a child per command, which matches the assignment architecture.

Comprehensive runtime outputs for every C1–C4 requirement are not preserved in the conversation. The assignment itself provides examples such as `printf ... | sort` and redirection combinations, but those examples are specification examples, not evidence that the user ran them.

One important evidence distinction is that the project was compiled under `-Wall -Wextra -Werror` and later generated executor variants were subjected to actual compilation. The transcript includes a real misleading-indentation compiler failure in `executor.c` during F2 integration, demonstrating that generated executor code was concretely compiled and not merely stored.

The central workflow visible in the conversation is therefore iterative rather than linear: requirements/context were supplied; AI produced implementation artifacts; the user integrated them; the project was compiled; runtime behavior was observed; actual errors were reported to AI; AI diagnosed and supplied a correction; and the user retested. F2 provides the clearest complete example of this cycle.

Part C — File Redirection and Pipes — Contributions, Evidence, Reflection and Screenshots

7. My contribution vs AI contribution

User: supplied and integrated the project baseline, ran builds, and incorporated generated executor versions. The user also directly edited files using shell commands during later debugging.

AI: generated multiple executor and redirection/process/job-control artifacts, including the modular executor set recorded as generated at user turn 28.

Shared/iterative: AI proposed implementations; the user compiled and supplied compiler/runtime feedback; subsequent AI responses diagnosed and corrected issues.

8. Evidence available for submission

Assignment Part C: Pasted markdown(4).md, lines 440–566.

Baseline executor.c is a user-uploaded source file; visible source includes resolve_binary_path(), handle_input_redirections(), handle_output_redirections() and execute_pipeline().

Generated modular artifacts at user turn 28 include executor.c/h, executor_utils.c/h, redirection.c/h, process.c/h and jobcontrol.c/h.

Later F2 compiler output demonstrates actual compilation of src/executor.c under -Werror and the resulting diagnostic.

9. Reflection

Part C shows where direct AI code generation can be particularly valuable: process and descriptor management involves many interacting system calls and error paths. At the same time, it is also where generated code needs careful validation. The existence of multiple executor revisions and later compiler fixes is evidence that code generation accelerated development but did not eliminate the need for compilation, inspection and correction.

10. Suggested screenshots

- Assignment Part C requirements.
- Baseline executor.c showing command resolution.
- executor.c showing input-redirection logic.
- executor.c showing output-relay logic.
- Generated modular executor/redirection/process file cards.
- Any build output showing generated executor files compiling.

11. ChatGPT conversation link placement

For submission, the ChatGPT conversation link should be placed beside the evidence index entry for this section. The link should point to the actual conversation containing the relevant prompt/response and, where possible, the generated file card or debugging exchange. Do not create a separate link claiming to represent a conversation that does not exist. For early sections where the exact prompt is unavailable in the current evidence, the report should rely on the actual conversation/file artifact that is available.

Part D — Sequential and Background Execution

Evidence status: Implementation/file evidence is available; exact early prompts and/or complete runtime transcripts are not always preserved.

Part D adds sequential execution with `;` and background execution with `&`. Sequential execution requires waiting for each command and stopping the sequence only when a command cannot be started. Background execution requires independent launches, monotonically increasing job numbers, immediate prompts, `SIGCHLD`-based reaping, correct completion messages and terminal-input protection.

This part depends heavily on the executor and process-management architecture. The generated `executor_d1.c` and `executor_d2.c` artifacts in the conversation file inventory provide direct evidence that AI generated dedicated implementation variants for these two stages. This is stronger AI-use evidence than a generic statement that AI 'helped with job control'.

The assignment also notes that terminal-control functionality from E1 may be needed for background jobs that attempt terminal input. The later generated `process/jobcontrol` artifacts therefore form part of the implementation chain.

1. Section overview and project role

The assignment specification assigns this part a defined set of behaviors and constraints. In the overall shell, the part is not isolated: it consumes parser structures, depends on executor behavior, and can affect later process-management features. The implementation evidence therefore has to be read as part of the evolving shell architecture rather than as a collection of independent snippets.

The AI-use question is separate from the implementation question. A source file may be correct without being AI-generated, and an AI-generated file may require substantial human correction before it becomes usable. This report keeps those questions separate throughout.

2. AI involvement and available prompt evidence

The user stated that the project was developed with generous AI use and that the dominant interaction style was to provide requirements/specification and ask ChatGPT to generate required files directly. The accessible project evidence supports that general description through multiple generated artifacts. However, for several early sections the exact original prompt text is not present in the retrievable conversation record. For those cases, this report does not fabricate a quotation. The evidence is instead described using the actual generated-file metadata and the source code that resulted.

Where a prompt is directly visible in the current conversation, it is quoted only when it actually appears. For F2, for example, the conversation contains explicit requests such as “edit this” followed by the full `snoop.c` source, and later explicit testing instructions and results. For earlier sections, the evidence is weaker at the prompt-text level and stronger at the artifact level.

Part D — Sequential and Background Execution — Implementation and AI-Generated Work

3. AI-generated implementation

D1's essential design is to iterate over command groups in order, invoke the pipeline executor for each foreground sequence, and stop only when the shell cannot start a command. The executor infrastructure records a failed-to-start condition separately from a non-zero exit status, matching the assignment's distinction between failure to execute and a program that executes and returns non-zero.

D2 requires a background launcher that forks without waiting, assigns a session-wide job number, tracks the process group and immediately returns to the prompt. The project later uses process groups and SIGCHLD handling. The generated `executor_d2.c` is direct evidence that AI was asked to produce a dedicated background-execution stage.

The modular process/jobcontrol package generated later contains `process.c/h` and `jobcontrol.c/h`. This separation is consistent with the assignment's requirement for a modular project and provides a reusable abstraction for job tracking, process groups and terminal state.

4. How the implementation maps to the specification

The implementation should be understood as a mapping from explicit requirements to concrete data structures, system calls and control flow. The strongest examples in the available evidence are the parser structures in A, the built-in source files in B, the executor/redirection infrastructure in C, generated D1/D2 executor stages, generated E1 process/job-control infrastructure, and the F1/F2 patches. In each case, the AI contribution is best described as generating or revising concrete implementation artifacts against a supplied specification.

A significant benefit of AI in this setting was the ability to produce complete multi-file changes rather than isolated code fragments. The conversation file inventory records generated artifacts at multiple stages, including executor variants and a modular package. This matters because the assignment explicitly penalizes monolithic code and expects functionality to be separated across `.c/.h` files.

At the same time, the implementation evidence contains imperfections. The parser source visible in the conversation contains an "Invalid syntax!" message even though the assignment requires "cshell: invalid syntax". The initial snoop implementation failed to compile because `environ` was undeclared and later had runtime `ptrace` issues. These concrete imperfections demonstrate why generated code still requires engineering review.

Part D — Sequential and Background Execution — Development Workflow and Verification

5. Development workflow

The file inventory gives a concrete timeline: `executor_d1.c` was generated at user turn 17; `executor_d2.c` at user turn 21; `executor_e1.c` at user turn 26; and the modular `executor/process/jobcontrol` package at user turn 28. These artifacts demonstrate progressive AI-generated implementation rather than a single one-shot answer.

The user integrated these artifacts and later used background jobs as part of testing. For example, during F2 attach-mode testing the user executed `'sleep 30 &'` and obtained a job/PID from the shell before running `'snoop -p 28046'`. The shell output `'[2] 28046'` and later process completion behavior are concrete evidence that background execution and job tracking were present in the tested shell.

At one point a shorter `'sleep 5 &'` process exited before `snoop -p` was attempted. The user supplied that output, and the AI correctly interpreted it as a timing issue: the target process no longer existed. This is a documented example of using actual runtime behavior to reason about the process model.

6. Verification and testing

The strongest D-related runtime evidence in the accessible conversation is the background-job behavior used for F2. The user ran `'sleep 5 &'` and the shell reported `'[1] 27805'`, followed by `'sleep with pid 27805 exited normally'`. The subsequent `snoop -p` attempt returned `'snoop: no such process'`, consistent with the process having already terminated.

The later test used `'sleep 30 &'` and produced `'[2] 28046'`, after which `snoop -p` successfully attached and traced the process. This demonstrates that the shell could launch a background process, report a job number and PID, and continue accepting commands.

However, the transcript does not contain a complete dedicated D test suite for every D1/D2 requirement. The report therefore does not claim that every sequential/background edge case was verified.

The central workflow visible in the conversation is therefore iterative rather than linear: requirements/context were supplied; AI produced implementation artifacts; the user integrated them; the project was compiled; runtime behavior was observed; actual errors were reported to AI; AI diagnosed and supplied a correction; and the user retested. F2 provides the clearest complete example of this cycle.

Part D — Sequential and Background Execution — Contributions, Evidence, Reflection and Screenshots

7. My contribution vs AI contribution

User: integrated generated executor stages, compiled the project and ran background processes. The user supplied actual terminal output from those runs.

AI: generated `executor_d1.c` and `executor_d2.c`, later generated the modular process/jobcontrol infrastructure, and interpreted runtime behavior during debugging.

Not claimed: that the user manually wrote D1/D2 implementation from scratch; the generated artifact metadata directly supports AI code generation.

8. Evidence available for submission

Assignment Part D: Pasted markdown(4).md, lines 568–640.

Generated `executor_d1.c`: source_kind generated, user turn 17.

Generated `executor_d2.c`: source_kind generated, user turn 21.

Generated `process.c/h` and `jobcontrol.c/h`: source_kind generated, user turn 28.

Runtime evidence: user terminal output for 'sleep 5 &' and later 'sleep 30 &' with job/PID reporting.

9. Reflection

Part D is one of the clearest examples of the value of AI as a code-generation accelerator. Job control combines process lifecycle, waiting, signals and shell responsiveness. The generated D1/D2 files gave the project a concrete implementation starting point, while actual background-job behavior exposed timing and lifecycle realities that could not be established from code generation alone.

10. Suggested screenshots

- Assignment Part D requirements.
- Conversation file inventory showing generated `executor_d1.c` and `executor_d2.c`.
- Generated modular `process.c/jobcontrol.c` file cards.
- Terminal showing 'sleep 5 &' and the shell's job/PID output.
- Terminal showing 'sleep 30 &' followed by successful `snoop -p` attachment.
- Any sequential execution test screenshot if available.

11. ChatGPT conversation link placement

For submission, the ChatGPT conversation link should be placed beside the evidence index entry for this section. The link should point to the actual conversation containing the relevant prompt/response and, where possible, the generated file card or debugging exchange. Do not create a separate link claiming to represent a conversation that does not exist. For early sections where the exact prompt is unavailable in the current evidence, the report should rely on the actual conversation/file artifact that is available.

Part E — Exotic Shell Intrinsics

Evidence status: Implementation/file evidence is available; exact early prompts and/or complete runtime transcripts are not always preserved.

Part E contains activities, terminal control, resume and ping. It is the most process-management-heavy part of the assignment. E1 requires process groups and job listings; E2 requires terminal ownership and signal handling for Ctrl-C, Ctrl-Z and Ctrl-D; E3 resumes jobs in foreground/background modes with optional timeout; E4 sends signals to shell-tracked processes or process groups.

The conversation provides especially concrete evidence for E4. The user explicitly reported that ping was implemented and tested with PID targets, %job targets, signal-number modulo behavior, syntax errors and unknown targets. The generated executor/jobcontrol infrastructure also supports the surrounding E1–E3 functionality.

Some exact E1–E3 test transcripts are not present in the accessible conversation. Therefore the report distinguishes generated artifacts from verified runtime claims.

1. Section overview and project role

The assignment specification assigns this part a defined set of behaviors and constraints. In the overall shell, the part is not isolated: it consumes parser structures, depends on executor behavior, and can affect later process-management features. The implementation evidence therefore has to be read as part of the evolving shell architecture rather than as a collection of independent snippets.

The AI-use question is separate from the implementation question. A source file may be correct without being AI-generated, and an AI-generated file may require substantial human correction before it becomes usable. This report keeps those questions separate throughout.

2. AI involvement and available prompt evidence

The user stated that the project was developed with generous AI use and that the dominant interaction style was to provide requirements/specification and ask ChatGPT to generate required files directly. The accessible project evidence supports that general description through multiple generated artifacts. However, for several early sections the exact original prompt text is not present in the retrievable conversation record. For those cases, this report does not fabricate a quotation. The evidence is instead described using the actual generated-file metadata and the source code that resulted.

Where a prompt is directly visible in the current conversation, it is quoted only when it actually appears. For F2, for example, the conversation contains explicit requests such as “edit this” followed by the full snoop.c source, and later explicit testing instructions and results. For earlier sections, the evidence is weaker at the prompt-text level and stronger at the artifact level.

Part E — Exotic Shell Intrinsic — Implementation and AI-Generated Work

3. AI-generated implementation

The assignment requires every pipeline to run in its own process group, using `setpgid()` in both parent and child contexts. The generated process/jobcontrol architecture is designed around this model. The executor code visible in the conversation also contains `setpgid()`, terminal handoff and wait logic. This architecture supports activities and foreground/background transitions.

E2 relies on `tcsetpgrp()` to give the terminal to the foreground pipeline and reclaim it afterward. The assignment also requires the shell to ignore or handle `SIGINT/SIGTSTP/SIGTTOU` appropriately. The generated job-control modules are therefore directly relevant to E2, even though a full E2-specific transcript is not preserved.

E4 ping has a clear semantic contract. A numeric target is a PID, while `%N` denotes a tracked job. The typed signal number is validated first, then reduced modulo 64 for the actual signal, while the original number is echoed in the success message. The user explicitly tested the modulo behavior with 79 producing signal 15 and also tested PID/job targets and invalid cases.

4. How the implementation maps to the specification

The implementation should be understood as a mapping from explicit requirements to concrete data structures, system calls and control flow. The strongest examples in the available evidence are the parser structures in A, the built-in source files in B, the executor/redirection infrastructure in C, generated D1/D2 executor stages, generated E1 process/job-control infrastructure, and the F1/F2 patches. In each case, the AI contribution is best described as generating or revising concrete implementation artifacts against a supplied specification.

A significant benefit of AI in this setting was the ability to produce complete multi-file changes rather than isolated code fragments. The conversation file inventory records generated artifacts at multiple stages, including executor variants and a modular package. This matters because the assignment explicitly penalizes monolithic code and expects functionality to be separated across `.c/.h` files.

At the same time, the implementation evidence contains imperfections. The parser source visible in the conversation contains an “Invalid syntax!” message even though the assignment requires “cshell: invalid syntax”. The initial snoop implementation failed to compile because `environ` was undeclared and later had runtime `ptrace` issues. These concrete imperfections demonstrate why generated code still requires engineering review.

Part E — Exotic Shell Intrinsic — Development Workflow and Verification

5. Development workflow

The file inventory records `executor_e1.c` as generated at user turn 26, followed by `executor_e1_modular.zip` and a modular `executor/process/jobcontrol` set at user turn 28. This is direct evidence of AI-generated work around E1 and the infrastructure required for E2/E3.

For E4, the conversation records the user working with the ping implementation and explicitly reporting successful tests. The documented tests included normal PID use, `%job` use, modulo behavior (79 to 15), invalid syntax, and unknown targets. The assignment requires exactly these categories of behavior.

The workflow is iterative: generated code was integrated, compiled, executed, and then the user reported actual results. This same pattern later appears very clearly in F1/F2, where compiler diagnostics and `ptrace` runtime behavior were fed back to AI.

6. Verification and testing

The conversation establishes that E4 worked. The user reported normal PID tests, `%job` tests, modulo testing with 79→15, invalid syntax and unknown-target handling. These tests correspond closely to the explicit E4 requirements in the assignment.

The background-job tests also provide supporting evidence for the process-group/job infrastructure used by E1–E4. The shell printed job numbers and PIDs for background sleeps, and a later `snoop -p` successfully attached to one of those PIDs.

A complete transcript for activities output, Ctrl-Z terminal stopping, resume --timeout behavior, or Ctrl-D stopped-job handling is not available in the evidence used here. Those are therefore marked as not independently evidenced in this report.

The central workflow visible in the conversation is therefore iterative rather than linear: requirements/context were supplied; AI produced implementation artifacts; the user integrated them; the project was compiled; runtime behavior was observed; actual errors were reported to AI; AI diagnosed and supplied a correction; and the user retested. F2 provides the clearest complete example of this cycle.

Part E — Exotic Shell Intrinsic — Contributions, Evidence, Reflection and Screenshots

7. My contribution vs AI contribution

User: provided the E requirements, integrated generated executor/job-control artifacts, compiled and ran tests, and reported actual E4 outcomes.

AI: generated `executor_e1.c` and the modular executor/process/jobcontrol package, and provided implementation guidance for ping.

Verification: user-side execution is especially important because signal and terminal behavior depends on the real shell/process environment.

8. Evidence available for submission

Assignment Part E: Pasted markdown(4).md, lines 642–794, including E1–E4 requirements.

Generated `executor_e1.c` at user turn 26 and `executor_e1_modular.zip` plus modular process/jobcontrol files at user turn 28.

Conversation record: user reported E4 tests for PID, %job, modulo 79→15, invalid syntax and unknown targets.

Runtime support: 'sleep 30 &' produced [2] 28046, followed by successful snoop -p 28046.

9. Reflection

E demonstrates why AI assistance is useful for systems programming but also why verification matters. Process groups, terminal ownership and signals have subtle interactions that are difficult to validate from static code alone. The strongest documented E evidence is E4's test matrix and the generated job-control artifacts. The report intentionally avoids claiming unrecorded E2/E3 test results.

10. Suggested screenshots

- Assignment E1–E4 requirements.
- Generated `executor_e1.c` artifact card.
- Generated `executor_e1_modular.zip` artifact card.
- Generated `process.c/jobcontrol.c` artifact cards.
- Terminal showing the E4 PID/%job/modulo/error tests, if the earlier chat portion is captured.
- Terminal showing background job creation used for process-group infrastructure.

11. ChatGPT conversation link placement

For submission, the ChatGPT conversation link should be placed beside the evidence index entry for this section. The link should point to the actual conversation containing the relevant prompt/response and, where possible, the generated file card or debugging exchange. Do not create a separate link claiming to represent a conversation that does not exist. For early sections where the exact prompt is unavailable in the current evidence, the report should rely on the actual conversation/file artifact that is available.

Part F — Fun Stuff

Evidence status: Strong direct AI-generation and runtime evidence is available.

Part F contains spy and snoop. These features are explicitly operating-system-oriented: spy inspects process open files through /proc, while snoop traces syscalls using ptrace. The conversation provides the most detailed and direct AI-usage evidence for this section because the user explicitly asked AI to generate files, installed generated patches, compiled them, reported failures, and iteratively applied corrections.

F1 requires spy [pid], including shell inspection when no PID is given, PID/FD/TYPE/PATH output, cwd/txt/mem/numeric descriptors, correct file types, deleted-file handling, nonexistent-process handling and invalid syntax. F2 requires snoop command mode and -p attach mode, PTRACE_TRACEME/PTRACE_ATTACH, PTRACE_SYSCALL entry/exit timing, aggregation, sorting, unknown syscall fallback and exact error messages.

This section therefore serves as the strongest case study for extensive AI code generation combined with human integration and validation.

1. Section overview and project role

The assignment specification assigns this part a defined set of behaviors and constraints. In the overall shell, the part is not isolated: it consumes parser structures, depends on executor behavior, and can affect later process-management features. The implementation evidence therefore has to be read as part of the evolving shell architecture rather than as a collection of independent snippets.

The AI-use question is separate from the implementation question. A source file may be correct without being AI-generated, and an AI-generated file may require substantial human correction before it becomes usable. This report keeps those questions separate throughout.

2. AI involvement and available prompt evidence

The user stated that the project was developed with generous AI use and that the dominant interaction style was to provide requirements/specification and ask ChatGPT to generate required files directly. The accessible project evidence supports that general description through multiple generated artifacts. However, for several early sections the exact original prompt text is not present in the retrievable conversation record. For those cases, this report does not fabricate a quotation. The evidence is instead described using the actual generated-file metadata and the source code that resulted.

Where a prompt is directly visible in the current conversation, it is quoted only when it actually appears. For F2, for example, the conversation contains explicit requests such as “edit this” followed by the full snoop.c source, and later explicit testing instructions and results. For earlier sections, the evidence is weaker at the prompt-text level and stronger at the artifact level.

Part F — Fun Stuff — Implementation and AI-Generated Work

3. AI-generated implementation

For F1, the conversation records that the user downloaded '/home/sankeerth/Downloads/f1_spy_patch.zip', extracted it to /tmp/f1_patch and copied spy.c, spy.h and executor_f1.c into the project. The initial spy implementation crashed in print_memory_maps(). The AI then supplied a safer replacement implementation. A further edge case occurred when a deleted regular file was classified incorrectly; AI suggested recognizing deleted absolute-path targets as REG when stat() could no longer succeed.

For F2, the AI generated a patch archive named f2_snoop_patch.zip containing snoop.c, snoop.h and executor_f2.c. The generated snoop.c used ptrace, waitpid, user registers, monotonic timestamps, a syscall-name table, dynamic statistics and qsort sorting. The executor integration added snoop_builtin() to the standalone-built-in path.

The generated tracer maintains an ActiveCall with syscall number and entry timestamp, detects syscall stops using SIGTRAP|0x80, reads orig_rax through PTRACE_GETREGS, computes elapsed time, aggregates counts, sorts by calls descending and first_seen, and falls back to syscall_N for unknown numbers. It also handles process termination so exit_group can be counted even without a conventional syscall-exit stop.

4. How the implementation maps to the specification

The implementation should be understood as a mapping from explicit requirements to concrete data structures, system calls and control flow. The strongest examples in the available evidence are the parser structures in A, the built-in source files in B, the executor/redirection infrastructure in C, generated D1/D2 executor stages, generated E1 process/job-control infrastructure, and the F1/F2 patches. In each case, the AI contribution is best described as generating or revising concrete implementation artifacts against a supplied specification.

A significant benefit of AI in this setting was the ability to produce complete multi-file changes rather than isolated code fragments. The conversation file inventory records generated artifacts at multiple stages, including executor variants and a modular package. This matters because the assignment explicitly penalizes monolithic code and expects functionality to be separated across .c/.h files.

At the same time, the implementation evidence contains imperfections. The parser source visible in the conversation contains an “Invalid syntax!” message even though the assignment requires “cshell: invalid syntax”. The initial snoop implementation failed to compile because environ was undeclared and later had runtime ptrace issues. These concrete imperfections demonstrate why generated code still requires engineering review.

Part F — Fun Stuff — Development Workflow and Verification

5. Development workflow

The F1 workflow was concrete and iterative. The user installed the patch, ran it, obtained a segmentation fault in `print_memory_maps()`, used `gdb` to locate the crash, and then replaced `spy.c` with a safer implementation supplied by AI. The user then tested `spy` on the shell and a background process, including a redirected/deleted file case and required error messages.

For F2, the user installed the generated patch and initially encountered a compilation error: 'environ undeclared'. AI instructed the user to add 'extern char **environ;'. A later build exposed a misleading-indentation error in `executor.c` caused by a duplicated/incomplete if line. The user displayed lines 42–65 with `nl -ba`; AI identified the duplicate; the user removed it with `sed -i`.

The first runtime test produced 'execve 1 0.001s' followed by 'cshell: command not found (snoop)'. AI diagnosed `executor` control flow. After an explicit snoop dispatch fix, the command no longer fell through. A second runtime issue then appeared: only `execve` was recorded because plain SIGTRAP was being forwarded to the tracee. AI replaced `traceprocess()` with logic that resumes plain SIGTRAP with signal 0 and accounts for an active terminating syscall.

After those corrections, 'snoop sleep 1' produced a detailed syscall summary including `mmap`, `read`, `openat`, `close`, `rt_sigaction`, `fstat`, `mprotect`, `statx`, `brk`, `fcntl`, `sigaltstack`, `access`, `pread64`, `prlimit64`, `munmap`, `execve`, `arch_prctl`, `setxattr`, `set_robust_list`, `getrandom`, `poll`, `sched_getaffinity`, `gettid`, `ioctl`, an unknown syscall_230 taking 1.000s, and `exit_group`. The user then tested `snoop -p` using 'sleep 30 &' and PID 28046; `attach` succeeded and produced a summary.

6. Verification and testing

F1 verified shell inspection with `spy`, process inspection with a background sleep, deleted-file handling, nonexistent PID and invalid syntax. The user reported that F1 tests passed.

F2 command mode was verified with 'snoop sleep 1'. The output contained multiple syscall counts, timing, sorting and `exit_group`. The 1.000s syscall demonstrated that the traced process remained alive for the duration of sleep.

F2 attach mode was verified with 'sleep 30 &' followed by 'snoop -p 28046'. The tracer produced a summary including `lsetxattr 1 23.794s`, `sigaltstack`, `munmap` and `exit_group`.

Error tests were also explicitly reported as working: 'snoop' produced 'snoop: invalid syntax', 'snoop definitely_not_a_command' produced 'snoop: command not found', and a nonexistent PID produced 'snoop: no such process'.

These are the strongest runtime evidence items in the entire AI-usage report because they are actual user terminal outputs recorded in the conversation.

The central workflow visible in the conversation is therefore iterative rather than linear: requirements/context were supplied; AI produced implementation artifacts; the user integrated them; the project was compiled; runtime behavior was observed; actual errors were reported to AI; AI diagnosed and supplied a correction; and the user retested. F2 provides the clearest complete example of this cycle.

Part F — Fun Stuff — Contributions, Evidence, Reflection and Screenshots

7. My contribution vs AI contribution

User: downloaded/extracted/copied patches, ran gdb, edited files with shell commands, compiled with make, ran the shell, created test processes, supplied exact compiler/runtime output, and decided whether the observed behavior was acceptable.

AI: generated the safer F1 implementation, the F2 snoop.c/snoop.h/executor_f2.c patch, diagnosed compiler and runtime failures, and supplied exact code replacements for traceprocess() and executor integration.

Joint iteration: the final implementation emerged through repeated cycles of generated code, real compilation, real runtime behavior, diagnosis and correction.

8. Evidence available for submission

F1 patch integration: user copied /tmp/f1_patch/spy.c, spy.h and executor_f1.c into src/include.

F1 debugging: gdb reported SIGSEGV in print_memory_maps(); AI supplied safer spy implementation.

F2 generated archive: assistant created /mnt/data/f2_snoop_patch.zip containing snoop.c, snoop.h and executor_f2.c.

Compiler evidence: user reported 'environ undeclared' and later the misleading-indentation diagnostic.

Runtime evidence: user output for snoop sleep 1, snoop -p 28046, invalid syntax and command-not-found tests.

Assignment F1/F2 requirements: Pasted markdown(4).md, lines 796–864.

9. Reflection

F is the clearest demonstration that extensive AI use can still involve substantial human verification. The AI generated major implementation artifacts, but those artifacts were not accepted blindly: compilation failed, executor flow failed at runtime, ptrace signal handling failed at runtime, and syscall termination accounting needed correction. The user repeatedly executed the shell and supplied exact outputs. This process is important evidence of responsible AI-assisted engineering rather than unverified code dumping.

10. Suggested screenshots

- F1 assignment requirements.
- User command showing F1 patch extraction/copy.
- gdb screenshot/output showing SIGSEGV in print_memory_maps().
- spy successful output for shell/background process.
- F2 patch download/file card showing snoop.c, snoop.h and executor_f2.c.
- Compiler output showing environ undeclared.
- executor.c lines 42–65 showing the duplicated if and subsequent corrected version.
- First failed snoop runtime output showing execve plus command-not-found.
- Final snoop sleep 1 output with detailed syscall summary.
- Final snoop -p 28046 output.
- Invalid syntax and command-not-found test outputs.

11. ChatGPT conversation link placement

For submission, the ChatGPT conversation link should be placed beside the evidence index entry for this section. The link should point to the actual conversation containing the relevant prompt/response and, where possible, the

generated file card or debugging exchange. Do not create a separate link claiming to represent a conversation that does not exist. For early sections where the exact prompt is unavailable in the current evidence, the report should rely on the actual conversation/file artifact that is available.

SECTION G — Overall AI Usage Summary

The project used AI as an extensive development assistant, with the strongest documented mode being direct code/file generation followed by user-side integration and verification. The conversation file inventory records a baseline project archive and source files supplied by the user, followed by multiple artifacts explicitly marked as generated. These include `executor_modified.c`, `executor_compact.c`, `executor_d1.c`, `executor_d2.c`, `executor_e1.c`, `executor_e1_modular.zip`, and a modular set containing `executor.c/h`, `executor_utils.c/h`, `redirection.c/h`, `process.c/h` and `jobcontrol.c/h`.

The later F1/F2 work makes the workflow especially explicit. For F1, the user installed a patch, encountered a SIGSEGV in `print_memory_maps()`, used `gdb`, and then applied an AI-supplied safer implementation. For F2, the assistant generated a zip containing `snoop.c`, `snoop.h` and `executor_f2.c`. The user compiled the code, reported an undeclared `environ` error, fixed it, encountered a misleading-indentation error in `executor.c`, displayed the relevant lines, removed the duplicate line, and continued testing. Runtime failures then exposed `executor` fall-through and `ptrace` SIGTRAP handling problems. AI supplied targeted corrections, and the user retested until `snoop` command and `attach` modes worked.

The overall contribution split should therefore be stated as follows: AI generated substantial implementation artifacts and debugging corrections; the user supplied the project/specification context, integrated those artifacts into the local project, compiled them, ran the shell, inspected compiler/runtime results, edited files when instructed, and made acceptance decisions based on observed behavior. The evidence does not support saying that every line of the project was generated by AI, because the initial source files are recorded as user uploads.

AI accelerated the project most strongly in areas with repetitive systems-programming scaffolding and complex API interactions: `executor` integration, process groups, redirection, job control, `/proc` inspection and `ptrace` tracing. Its limitations were also visible: generated code contained compilation errors, control-flow bugs and `ptrace` behavior that required correction. The project therefore demonstrates both the productivity benefit and the verification burden of AI-assisted programming.

A final submission should attach the strongest screenshots rather than every available message. The best evidence set is: generated-file metadata, the F1 `gdb` failure and correction, the F2 generated patch, compiler diagnostics, corrected `executor` lines, final `snoop` outputs, and the E4 test evidence. For A–D, source/file evidence and the generated `executor` artifacts are useful, but the report should explicitly acknowledge where a complete prompt/test transcript is unavailable.

SECTION I — AI Usage Declaration

I declare that artificial intelligence was used extensively during the development of the C Shell mini-project. My primary mode of AI use was to provide the project requirements/specification and project context to ChatGPT and request direct generation or modification of implementation files. AI assistance was used across multiple parts of the project, particularly for executor integration, process management, job control, process inspection and syscall tracing.

I further declare that the AI-generated material was not treated as automatically correct. I integrated generated files into the project, compiled the resulting code, executed the shell, inspected compiler diagnostics and runtime output, and supplied observed failures back to ChatGPT for diagnosis. The F1 and F2 development record provides direct examples: a segmentation fault in spy was investigated with gdb; snoop initially failed to compile because environ was undeclared; executor integration produced a misleading-indentation compiler error; snoop initially fell through to normal command execution; and ptrace signal handling caused premature termination. These issues were identified through actual execution and corrected iteratively.

I distinguish between AI-generated work and my own work. AI generated substantial code artifacts and patches, including multiple executor revisions and the snoop implementation/patch. I supplied the requirements and project files, integrated the generated artifacts, ran the build and test commands, inspected outputs, edited files when required, and determined whether observed behavior matched the requirements. The initial project source files that are recorded as user uploads are not represented in this declaration as AI-generated merely because AI later worked on related components.

Where the exact wording of an earlier AI prompt is no longer available in the evidence record, I have not reconstructed or fabricated it. Similarly, I do not claim runtime tests that are not documented in the conversation. The purpose of this report is to provide a truthful account of AI-assisted development, including both the extent of AI contribution and the verification work required to turn generated code into a functioning project.

This declaration is intended to accompany the relevant ChatGPT conversation link(s), generated file artifacts, source-code screenshots and terminal screenshots as evidence of AI usage.

Student signature: _____

Date: _____

Appendix — High-Confidence Conversation Evidence

F1 patch integration: the user reported downloading /home/sankeerth/Downloads/f1_spy_patch.zip, extracting it to /tmp/f1_patch and copying spy.c, spy.h and executor_f1.c into the project. This is direct evidence of artifact integration.

F1 crash: the user reported a gdb result showing SIGSEGV in print_memory_maps(). AI then supplied a safer implementation. This is direct debugging evidence.

F2 generated patch: AI created f2_snoop_patch.zip containing snoop.c, snoop.h and executor_f2.c. The archive was offered as a downloadable generated artifact in the conversation.

Compiler issue: the user reported: src/snoop.c:320:28: error: 'environ' undeclared. AI instructed adding extern char **environ;.

Executor issue: the user displayed executor.c lines 42–65. The output showed a duplicate incomplete if at line 50. AI identified it; the user ran sed -i '50d' src/executor.c.

First snoop runtime failure: 'snoop sleep 1' produced execve 1 0.001s followed by cshell: command not found (snoop). AI diagnosed the executor fall-through and supplied an explicit snoop dispatch fix.

ptrace runtime failure: after executor correction, snoop sleep 1 produced only execve. AI supplied a replacement traceprocess() that treats plain SIGTRAP as a ptrace event and accounts for terminating syscalls.

Successful command-mode trace: the user reported a detailed summary for snoop sleep 1, including mmap 27, read 14, openat 10, multiple other calls, syscall_230 at 1.000s, and exit_group 1.

Successful attach-mode trace: the user ran sleep 30 &, received [2] 28046, then ran snoop -p 28046 and obtained a summary including lsetxattr 1 23.794s, sigaltstack, munmap and exit_group.

Error tests: the user explicitly stated that snoop, nonexistent-command and other error tests worked.

These high-confidence items should be prioritized when selecting screenshots for the college submission because they directly connect AI-generated code to actual development, debugging and validation.